

Linguagem de Programação I

Prof. Orlando Saraiva Júnior
orlando.nascimento@fatec.sp.gov.br

**Como um time trabalha no mesmo
código, ao mesmo tempo, sem
sobrescrever o trabalho um do outro?**

Um problema comum antes do git

Um clássico que todo desenvolvedor já viveu (ou vai viver):



```
projeto_final.zip  
projeto_final_v2.zip  
projeto_final_v2_CORRIGIDO.zip  
projeto_final_v2_CORRIGIDO_usar_esse.zip  
projeto_final_v2_CORRIGIDO_usar_esse_FINAL.zip  
# qual desses tem o código mais atualizado?
```

Controle de versão existe para responder, com certeza, a essa pergunta.

O que o git guarda ?

Desfazendo um mito comum

O Git não guarda apenas “as diferenças entre arquivos”. A cada commit, ele guarda um snapshot (uma fotografia completa) do estado do projeto naquele momento.

- **Cada commit** aponta para o commit anterior, formando uma cadeia — o histórico do projeto.
- **Isso é o que permite** voltar a qualquer ponto no tempo, comparar versões, e criar branches sem custo real.
- **Um arquivo idêntico entre dois commits** nem é duplicado — o Git é inteligente o suficiente para reaproveitar o que não mudou.

O Git foi criado por Linus Torvalds — o mesmo criador do Linux — em apenas alguns dias, em abril de 2005. O motivo foi uma crise real de colaboração: o kernel Linux usava até então uma ferramenta proprietária chamada BitKeeper, e quando a empresa retirou a licença gratuita usada pela comunidade, Torvalds precisou de uma alternativa rápida.

O requisito era ambicioso

Um sistema distribuído, rápido, capaz de lidar com milhares de colaboradores trabalhando em paralelo no mesmo projeto — exatamente o tipo de escala que o desenvolvimento do kernel Linux exige.

Hoje, quase 20 anos depois, o Git é o sistema de controle de versão mais usado do mundo.



```
$ git init
# inicia um repositório Git na pasta atual

$ git status
# mostra o que mudou desde o último commit

$ git add arquivo.py
# prepara o arquivo para entrar no próximo commit (staging area)

$ git commit -m "feat: adiciona validação de login"
# grava um snapshot permanente do projeto, com uma mensagem

$ git log
# mostra o histórico de commits

$ git diff
# mostra exatamente o que mudou, linha por linha
```

Sincronizando com o Remoto



```
$ git clone https://github.com/usuario/projeto.git
# copia um repositório remoto inteiro para sua máquina

$ git push origin minha-branch
# envia seus commits locais para o repositório remoto

$ git pull origin main
# traz e já aplica as mudanças do remoto na sua branch atual
```



pull x fetch — a diferença que confunde iniciantes

git fetch busca as mudanças do remoto mas não aplica nada — é “olhar sem tocar”. git pull faz um fetch e, em seguida, tenta um merge automático. Em times, fetch + revisão manual é mais seguro.

Modelo mais estruturado, com branches de longa duração para cada finalidade. Mais adequado a ciclos de release espaçados e produtos com versões bem definidas.



main

Código em produção, sempre estável.



develop

Integração contínua das próximas funcionalidades.



feature/*

Uma branch por funcionalidade, saindo de develop.



release/*

Preparação final de uma versão antes de ir para produção.



hotfix/*

Correção urgente direto em produção.

GitHub: da Ferramenta à Plataforma

Git é a ferramenta. GitHub é a plataforma social e colaborativa construída em torno dela.

A partir daqui, o foco sai de comandos individuais e vai para como um time se comunica e colabora através do código.

Pull Request

O principal ponto de comunicação do time

Um Pull Request não é só “juntar código” — é o principal ponto de comunicação assíncrona e feedback contínuo de um time de desenvolvimento.

Mais Boas Práticas

`.gitignore`

Lista do que nunca deve ir para o repositório: credenciais, chaves de API, node_modules, arquivos de build gerados automaticamente.

Commits Pequenos e Frequentes

Um commit por mudança lógica, não um commit gigante ao fim do dia.

Erro comum

Comitar um arquivo `.env` com senhas reais é uma das causas mais comuns de vazamento de credenciais em produção — configure o `.gitignore` antes do primeiro commit, não depois.

Dúvidas

Prof. Orlando Saraiva Júnior
orlando.nascimento@cps.sp.gov.br

Vamos fazer uma dinâmica prática



Tempo para Atividade